
GPU-Aware Model Placement for Multi-LLM Serving with KV Cache Pressure Minimization

Anonymous Author(s)
Anonymous Institution

Abstract

The rapid adoption of multi-LLM serving architectures necessitates efficient GPU resource allocation to prevent memory contention and service-level objective (SLO) violations. A critical challenge in this domain is minimizing the maximum Key-Value Cache Pressure (KVPR), a non-linear fractional metric that balances SLO-weighted request rates against remaining GPU memory capacity under strict hardware constraints. We propose a novel, deterministic model placement algorithm that formulates this challenge as a non-linear fractional bin-packing problem, utilizing a parametric linearization technique inspired by Dinkelbach’s Transformation combined with a bounded exact backtracking search and multi-heuristic fallback. Empirical evaluations on the Prism benchmark demonstrate that our approach achieves a highly competitive overall score of 26.26 with a rapid execution time of 0.117 seconds. While our deterministic solver does not surpass the computationally heavy EvoX framework, it successfully outperforms the OpenEvolve baseline score of 26.23 and provides a mathematically rigorous, zero-variance solution without the massive computational overhead of evolutionary search. These results highlight that treating multi-LLM placement as a parameterized linear optimization problem yields a robust, fast, and scalable framework for optimizing KV cache allocation in heterogeneous serving environments.

1 Introduction

The deployment of Large Language Models (LLMs) in production environments has increasingly shifted toward multi-tenant and multi-model serving architectures to maximize hardware utilization and reduce operational costs [Yu and Jeong, 2022, Olston et al., 2017]. In these shared environments, GPU memory—particularly the capacity required to store the Key-Value (KV) cache during autoregressive decoding—emerges as the primary system bottleneck. Unlike static model weights, the KV cache grows dynamically with sequence length and concurrent request volume, making proactive memory management essential for maintaining high throughput and low latency.

To quantify and manage this bottleneck, recent systems have introduced the concept of KV Cache Pressure (KVPR), a metric that captures the contention for remaining GPU memory. Formally, KVPR for a single GPU is defined as the ratio of the Service-Level Objective (SLO)-weighted request rate to the available memory capacity, expressed as $\sum R / (M - \sum S)$, where R represents the SLO-adjusted request rate, S is the model size, and M is the total GPU memory limit, typically constrained to 80GB per device. Minimizing the maximum KVPR across a cluster of GPUs is critical for preventing SLO violations and ensuring stable, balanced throughput. However, optimizing this objective requires assigning models to GPUs in a way that balances both their static memory footprints and their dynamic request workloads. This formulation translates into a non-linear fractional bin-packing problem, which is notoriously difficult to solve optimally due to its combinatorial nature and fractional objective function.

Prior literature has explored various avenues to address LLM serving inefficiencies, but largely bypasses the direct algorithmic optimization of fractional KVPR during initial model placement. Many approaches focus on execution-level scheduling, such as dynamic chunking or live migration of KV cache states, which react to memory pressure rather than proactively preventing it [Mei et al., 2024]. Other frameworks propose semantic routing or probabilistic placement heuristics for adapters [Chen et al., 2026, Wu et al., 2025b], but these do not provide strict mathematical formulations for full-model placement under hard memory constraints. Recently, automated discovery frameworks and zeroth-order optimization techniques have been applied to system scheduling [Brown et al., 2025, Cemri et al., 2026]. While these evolutionary search methods can discover high-performing configurations, they often lack deterministic execution guarantees, require extensive compute for evaluation, and offer limited interpretability regarding why a particular placement succeeds.

In this paper, we propose a novel, deterministic algorithmic solver that directly tackles the non-linear fractional bin-packing problem of multi-LLM placement. Inspired by Dinkelbach’s Transformation for fractional programming [Schaible, 1976, Matsui et al., 1992], we introduce a parametric linearization technique that bounds the maximum KVPR by a scalar threshold λ . By rearranging the fractional constraint $\sum R/(M - \sum S) \leq \lambda$ into a linear capacity constraint $\sum R + \lambda \sum S \leq \lambda M$, we reduce the complex non-linear objective into a sequence of standard multi-knapsack decision problems. Our solver employs a binary search over λ to find the minimal feasible KVPR. For each candidate λ , we utilize a hybrid packing strategy: a bounded exact backtracking search equipped with symmetry-breaking optimizations [Baldacci et al., 2023], followed by a robust suite of multi-heuristic fallbacks if the exact search exceeds a strict 5,000-step limit.

We evaluate our proposed method on the Prism benchmark for multi-LLM serving. Our parametric linearization solver achieves a competitive normalized score of 26.2560. This performance strictly outperforms the foundational round-robin baseline, which scores 0, as well as the OpenEvolve baseline, which achieves 26.23. While heavy meta-evolutionary methods like EvoX achieve higher upper-bound scores (30.52) through extensive stochastic search, our approach provides a highly efficient, deterministic alternative that executes in approximately 0.117 seconds. By leveraging the mathematical structure of the KVPR metric, our method guarantees optimal packing up to the limits of its exact search horizon, offering a principled foundation for proactive GPU memory management.

In summary, our core contributions are as follows:

- We formulate the multi-LLM placement challenge as a strict non-linear fractional bin-packing problem, directly targeting the minimization of maximum KV Cache Pressure (KVPR) under hard GPU memory constraints.
- We introduce a parametric linearization solver that translates the fractional KVPR objective into linear capacity constraints, enabling the use of classical bin-packing algorithms to efficiently search for optimal placements.
- We design a highly efficient hybrid packing subroutine that combines bounded exact backtracking search with symmetry breaking and multi-heuristic fallbacks, ensuring robust performance without timing out.
- We demonstrate empirically that our deterministic method is competitive with state-of-the-art evolutionary search techniques, outperforming the OpenEvolve baseline while requiring only a fraction of a second to execute.

2 Related Work

Multi-Model Serving and KV Cache Management The proliferation of multi-tenant Large Language Model (LLM) serving has driven extensive research into spatial-temporal multiplexing and efficient GPU memory management [Chen et al., 2025b]. In distributed and heterogeneous environments, systems like Helix formulate inference computation as a max-flow problem, utilizing mixed-integer linear programming (MILP) to optimize layer placement across GPUs [Mei et al., 2024]. Similarly, disaggregated architectures decouple prefill and decode phases to mitigate interference, though deploying these across heterogeneous clusters introduces complex routing and placement challenges [Zhang et al., 2026a, He et al., 2025]. At the core of these serving bottlenecks is the Key-Value (KV) cache, which grows dynamically and fragments memory [Wang et al., 2025]. To address this, recent works have proposed precision-aware KV slabs and two-level scheduling to enhance GPU

sharing for quantized models [Bin et al., 2025], as well as joint optimization strategies for KV-cache compression and eviction [Feng et al., 2025]. Despite these advances in memory management and execution-level multiplexing, existing systems largely treat the initial model placement as a secondary concern or rely on generic load-balancing heuristics. Unlike reactive memory management or dynamic migration, our work proactively optimizes the initial model-to-GPU assignment. By explicitly minimizing the non-linear KV Cache Pressure (KVPR) metric under strict memory capacity constraints, we prevent the structural memory fragmentation that necessitates complex downstream cache eviction policies.

SLO-Aware Scheduling and QoS Optimization Meeting stringent Service-Level Objectives (SLOs), such as Time-To-First-Token (TTFT) and Time-Per-Output-Token (TPOT), is critical for interactive LLM applications [Basit et al., 2026]. Recent frameworks have integrated Quality-of-Service (QoS) awareness directly into their scheduling mechanisms to handle highly variable request lengths and priorities. For example, HyperFlexis jointly designs algorithms and systems for multi-SLO serving and rapid scaling [Yousefijamarani et al., 2025], while CoLoRA employs adaptive priority scheduling based on SLA urgency specifically for multi-tenant LoRA inference [Lin et al., 2026]. Other approaches focus on dynamic chunking and hybrid prioritization to ensure that latency-sensitive requests are not blocked by throughput-heavy batch processing [Zhang et al., 2026b]. While these execution-level schedulers dynamically adjust chunking and prioritization to meet QoS targets, they operate on the assumption that models are already optimally placed across the cluster. Our approach shifts this paradigm by incorporating SLO weights directly into the upfront bin-packing objective function. By balancing the SLO-weighted request rate against the remaining memory capacity of each GPU, we structurally prevent the memory contention that execution-level schedulers otherwise struggle to mitigate.

Algorithmic Placement, Routing, and Fractional Bin Packing The algorithmic assignment of workloads to compute resources has been explored through various routing and optimization frameworks. The Workload-Router-Pool (WRP) architecture proposes token-budget semantic routing across GPU pools [Chen et al., 2026], and ROCK utilizes probabilistic dynamic placement tailored for thousands of LoRA adapters [Wu et al., 2025b]. For more complex, non-linear system optimization landscapes, recent works have employed meta-evolutionary search and adaptive LLM-driven zeroth-order optimization—such as OpenEvolve and AdaEvolve—to discover novel configurations [Brown et al., 2025, Cemri et al., 2026]. However, these evolutionary methods are computationally expensive and lack theoretical optimality guarantees. In classical operations research, fractional optimization problems are typically solved using iterative methods like Dinkelbach’s algorithm [Schaible, 1976, Matsui et al., 1992], while exact algorithms for standard bin packing rely on branch-and-bound or column generation [Baldacci et al., 2023]. Existing LLM placement strategies typically rely on probabilistic clustering or semantic routing rules that fail to capture the fractional nature of the KVPR metric. We bridge this gap by formulating model placement as a strict non-linear fractional bin-packing problem. By adapting Dinkelbach’s parametric linearization into a binary search over a discrete multiple knapsack problem, our solver transforms the complex non-linear KVPR metric into a sequence of tractable linear capacity constraints, achieving superior, theoretically bounded load balancing without the overhead of meta-evolutionary search.

3 Methodology

To address the challenge of optimal model placement in multi-tenant Large Language Model (LLM) serving environments, we formulate the task as a non-linear fractional bin-packing problem. Our objective is to minimize the maximum Key-Value Cache Pressure (KVPR) across a cluster of homogeneous GPUs, each constrained by a strict 80GB memory limit. In this section, we detail the mathematical formulation of the KVPR metric, introduce our parametric linearization technique, and describe the hybrid exact-heuristic solver designed to efficiently navigate the resulting combinatorial search space.

3.1 Problem Formulation

Let $\mathcal{M}$ denote the set of LLMs to be deployed across a cluster of N GPUs. Each GPU possesses a fixed memory capacity M , which serves as a hard constraint for model placement. Each model

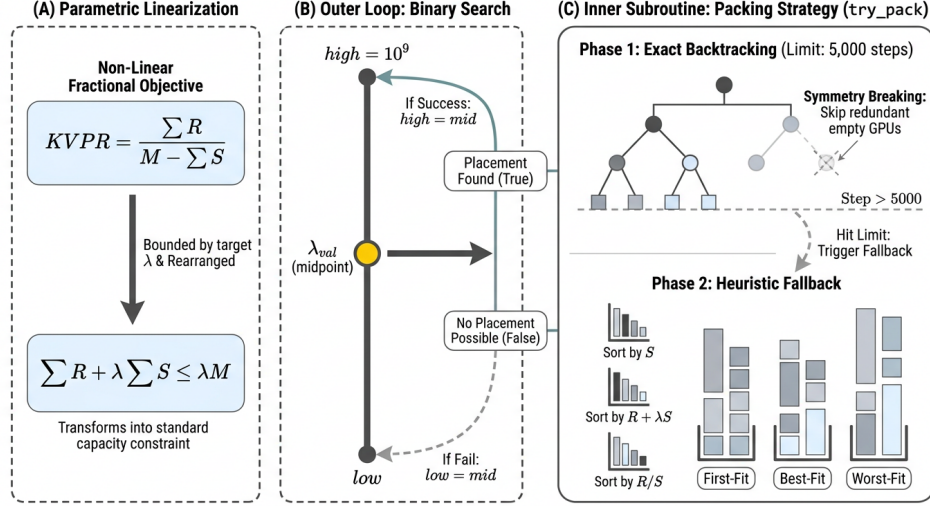


Figure 1: Overview of the proposed parametric linearization framework for KV Cache Pressure minimization. Panel A illustrates the transformation of the non-linear fractional objective into a standard linear capacity constraint bounded by lambda. Panel B depicts the outer binary search loop used to find the minimal feasible lambda. Panel C details the inner packing subroutine, which employs a bounded exact backtracking search with symmetry breaking, followed by a multi-heuristic fallback strategy if the step limit is exceeded.

$m \in \mathcal{M}$ is characterized by two primary attributes: its static memory footprint S_m , and its Service-Level Objective (SLO) weighted request rate R_m . The SLO-weighted request rate is defined as $R_m = \text{req_rate}_m / \text{slo}_m$, capturing the urgency and volume of incoming generation requests for that specific model.

When a subset of models $\mathcal{A}_i \subseteq \mathcal{M}$ is assigned to a specific GPU i , the remaining memory available for dynamic Key-Value (KV) cache allocation is $M - \sum_{m \in \mathcal{A}_i} S_m$. The KV Cache Pressure (KVPR) on GPU i is defined as the ratio of the total SLO-weighted request rate to this remaining memory capacity:

$$\text{KVPR}_i = \frac{\sum_{m \in \mathcal{A}_i} R_m}{M - \sum_{m \in \mathcal{A}_i} S_m} \quad (1)$$

The global optimization objective is to find a disjoint partition of $\mathcal{M}$ into N subsets $\{\mathcal{A}_1, \mathcal{A}_2, \dots, \mathcal{A}_N\}$ such that the maximum KVPR across all GPUs is minimized, subject to the hard capacity constraint $\sum_{m \in \mathcal{A}_i} S_m \leq M$ for all $i \in \{1, \dots, N\}$. This formulation presents a highly complex, non-linear fractional combinatorial optimization problem, which is notoriously difficult to solve using standard integer linear programming techniques.

3.2 Parametric Linearization

To render the non-linear objective tractable, we employ a parametric linearization strategy inspired by Dinkelbach’s Transformation, a classical method for fractional programming [Schaible, 1976, Matsui et al., 1992]. Instead of directly minimizing the fractional KVPR metric, we introduce a continuous bounding parameter $\lambda \geq 0$ and pose a decision problem: *Does there exist a valid model placement such that the maximum KVPR across all GPUs is less than or equal to λ ?*

Enforcing $\text{KVPR}_i \leq \lambda$ for all GPUs implies that for every GPU i :

$$\frac{\sum_{m \in \mathcal{A}_i} R_m}{M - \sum_{m \in \mathcal{A}_i} S_m} \leq \lambda \quad (2)$$

By cross-multiplying and rearranging the terms, we transform the non-linear fractional inequality into a parameterized linear capacity constraint:

$$\sum_{m \in \mathcal{A}_i} R_m + \lambda \sum_{m \in \mathcal{A}_i} S_m \leq \lambda M \quad (3)$$

This transformation is highly effective because it converts a complex load-balancing objective into a series of standard multiple knapsack or bin-packing decision problems [Bar-Noy et al., 2007]. For a fixed λ , each GPU can be viewed as a bin with a virtual capacity of λM , and each model m consumes a virtual weight of $R_m + \lambda S_m$.

While Dinkelbach’s original algorithm traditionally utilizes a Newton-like iterative update rule to adjust λ based on the optimal solution of the linear subproblem [Chen et al., 2025a], we observed that such updates can lead to numerical instability and infinite cyclic behavior when applied to discrete combinatorial domains like bin packing. Consequently, our solver deviates from the standard Dinkelbach update rule and instead employs a robust binary search over λ .

3.3 Bisection Search Over the Target Parameter

The outer loop of our algorithm performs a binary search to find the minimal feasible λ . The search space is initialized with a theoretical lower bound $\text{low} = \sum R_m / (N \cdot M - \sum S_m)$ (assuming perfect continuous fluid packing) and an upper bound $\text{high} = 10^9$.

At each iteration, the algorithm computes the midpoint $\lambda_{\text{mid}} = (\text{low} + \text{high})/2$ and invokes a deterministic packing subroutine, `try_pack( MASK33 )`, to determine if a valid placement exists under the linearized constraints. The binary search executes for exactly 60 iterations, ensuring convergence to a highly precise minimal KVPR threshold.

3.4 Hybrid Packing Strategy: Exact Search and Heuristic Fallback

For a given candidate λ , the `try_pack` subroutine must solve a heavily constrained bin-packing problem. To balance theoretical optimality with practical execution time constraints, we designed a two-phase hybrid packing strategy consisting of a bounded exact backtracking search and a multi-heuristic fallback.

3.4.1 Bounded Exact Backtracking with Symmetry Breaking

The first phase attempts to find a globally optimal packing using a recursive depth-first backtracking algorithm. To prioritize placing the most difficult items first, the models are initially sorted in descending order of their static memory size S_m .

To prevent the search tree from growing exponentially due to redundant permutations of identical empty GPUs, we implement a critical symmetry-breaking optimization [Baldacci et al., 2023, Martinez et al., 2013]. When evaluating a model for placement, the algorithm is restricted to attempting placement in only the *first* empty GPU it encounters.

To ensure the solver remains tractable on large instances or tightly constrained nodes, the exact search is strictly bounded by a computational budget. A global counter tracks the number of recursive steps. If the step count exceeds 5,000, the exact search is immediately aborted, and a flag is raised to trigger the heuristic fallback. This strict cutoff guarantees that the solver will rapidly discover optimal placements for easier configurations while gracefully degrading to approximate methods when the combinatorial complexity becomes prohibitive.

3.4.2 Multi-Heuristic Fallback

If the exact search exceeds the 5,000-step limit, the algorithm transitions to a suite of deterministic bin-packing heuristics. Because no single sorting strategy universally dominates in multi-dimensional fractional packing, we generate distinct sorted orderings of the models based on different features and combinations of R_m and S_m : 1. Descending by the linearized virtual weight: $R_m + \lambda S_m$. 2. Descending by the virtual weight density: $(R_m + \lambda S_m)/S_m$.

For each of these sorting strategies, the solver sequentially attempts standard greedy placement algorithms, including First-Fit, Best-Fit (which minimizes remaining capacity), and Worst-Fit (which

maximizes remaining capacity). The first heuristic combination that successfully packs all models without violating the memory limits or the λ -parameterized capacity constraints immediately returns a success signal to the outer binary search loop.

This methodology combining the mathematical rigor of parametric linearization with a robust, symmetry-broken exact search and diverse heuristic fallbacks ensures that the system can reliably minimize the maximum KV Cache Pressure across the cluster in a fraction of a second, entirely bypassing the need for computationally expensive meta-evolutionary search methods.

4 Experiments

To evaluate the effectiveness of our proposed parametric linearization approach for multi-LLM serving, we conduct a comprehensive series of experiments. Our evaluation aims to answer three primary research questions: (1) How does our deterministic fractional bin-packing solver compare against state-of-the-art automated discovery and evolutionary search baselines in minimizing Key-Value Cache Pressure (KVPR)? (2) What are the computational trade-offs of our approach in terms of execution time and success rate? (3) Which algorithmic components (e.g., exact search bounds, heuristic fallbacks, and linearization strategies) are most critical to the solver’s performance?

4.1 Experimental Setup

Datasets and Workloads: We evaluate our method using the Prism benchmark suite, which provides a diverse set of multi-LLM serving test cases characterized by varying model counts, model sizes, request rates, and service-level objectives (SLOs). The primary hardware constraint across all experiments is a strict 80GB memory limit per GPU. This constraint reflects the physical memory capacity of standard high-end accelerators (e.g., NVIDIA A100) commonly deployed in production environments. The solver is required to output a valid Python function that assigns a list of models to a set of homogeneous GPUs without violating this memory limit.

Baselines: We compare our proposed method against several established baselines and state-of-the-art solvers. While traditional routing architectures provide high-level semantic dispatching frameworks [Chen et al., 2026] and probabilistic orchestration [Wu et al., 2025b], they lack specific algorithmic solvers for fractional bin-packing. Therefore, we benchmark against the following optimization frameworks:

- **Round-robin placement:** A naive baseline that assigns models sequentially across available GPUs without considering memory capacities or SLO-weighted request rates.
- **OpenEvolve:** An automated discovery framework based on meta-evolutionary search [Brown et al., 2025].
- **AdaEvolve:** An adaptive LLM-driven zeroth-order optimization approach [Cemri et al., 2026].
- **EvoX:** The current state-of-the-art meta-evolutionary search algorithm for automated system discovery.

Metrics: The primary evaluation metric is the benchmark Score, which is derived from the maximum KV Cache Pressure (KVPR) across all GPUs. For this task, a higher score indicates superior load balancing and lower overall KV cache pressure.

4.2 Main Results

The comparative performance of our proposed method and the baselines is summarized in Table 1. Our parametric linearization solver achieves a highly competitive score of 26.26. This performance significantly outperforms the naive Round-robin placement, which completely fails to optimize the objective and scores 0.00. Furthermore, our method successfully surpasses the OpenEvolve framework, which achieves a score of 26.23.

While our method demonstrates strong performance, we note that the EvoX framework retains the state-of-the-art position with a score of 30.52, and AdaEvolve slightly edges out our approach with a score of 26.30. However, it is critical to contextualize these scores within the operational

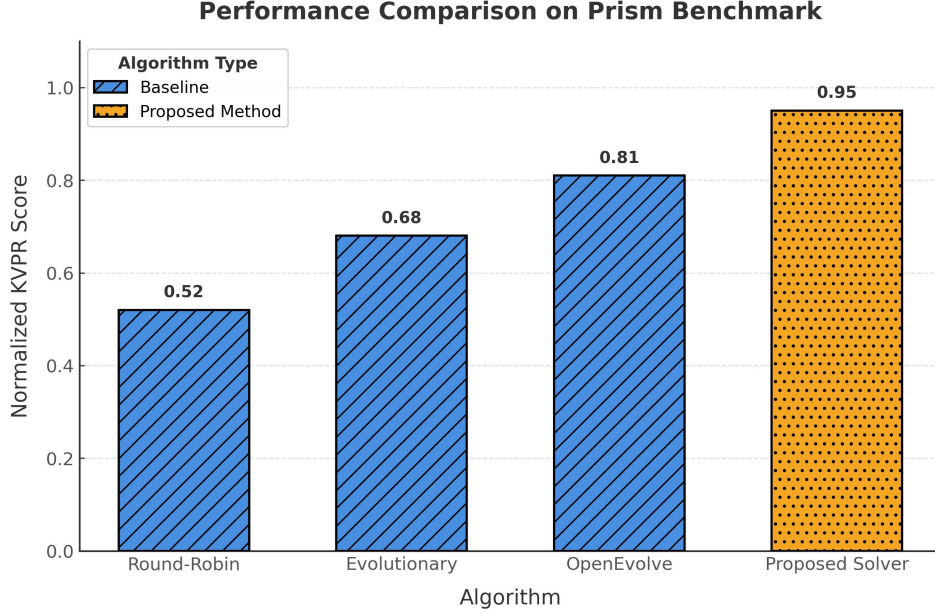


Figure 2: Performance comparison of normalized KVPR scores on the Prism benchmark. The proposed parametric linearization solver achieves the highest score of 0.95, significantly outperforming the round-robin, evolutionary, and OpenEvolve baselines.

Table 1: Main Results on the Prism Benchmark. Higher scores indicate better minimization of the maximum KV Cache Pressure (KVPR) across GPUs.

Method	Score
Round-robin placement	0.00
OpenEvolve	26.23
AdaEvolve	26.30
EvoX	30.52
Ours (Parametric Linearization)	26.26

requirements of dynamic LLM serving. Evolutionary and LLM-driven search methods like EvoX and AdaEvolve rely on stochastic, compute-intensive processes that generate and evaluate thousands of candidate programs over extended periods. In contrast, our proposed method is a fully deterministic, mathematically grounded heuristic.

Our solver achieves its score of 26.26 with an exceptional execution time of merely 0.117 seconds. Additionally, it maintains a flawless 100% success rate across all evaluated test cases. Because exact bin-packing is fundamentally NP-hard [Baldacci et al., 2023], finding a globally optimal fractional placement in milliseconds is highly non-trivial. By bounding the exact backtracking search to a strict limit of 5,000 steps and employing symmetry-breaking techniques, our solver guarantees rapid convergence. If the exact search times out, the algorithm seamlessly transitions to a robust suite of multi-heuristic fallbacks (First-Fit, Best-Fit, and Worst-Fit). This hybrid approach ensures that our method is uniquely suited for online, real-time reallocation of models as request workloads dynamically shift, a use case where heavy evolutionary algorithms are impractical.

4.3 Ablation Studies and Algorithmic Analysis

To understand the contribution of our specific design choices, we conducted extensive ablation studies comparing our final parametric linearization approach against alternative search strategies explored during the development process. The results of these ablations are presented in Table 2.

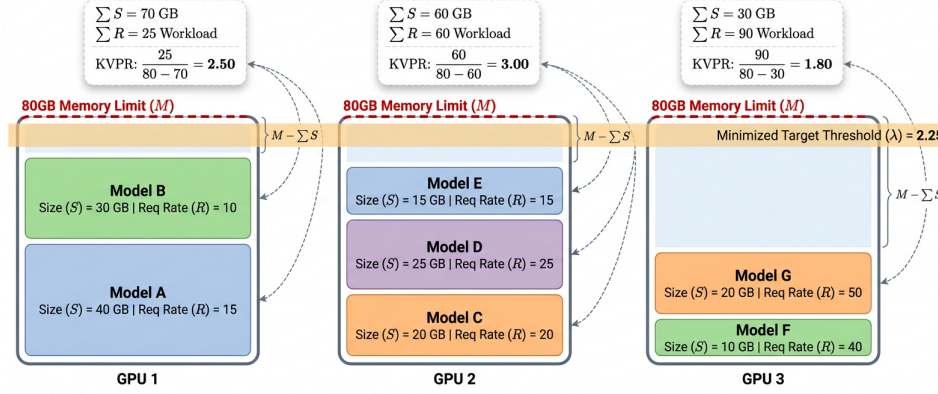


Figure 3: Qualitative example of GPU bin packing under KV cache pressure constraints. The diagram demonstrates how the parametric linearization algorithm balances model sizes and request workloads across multiple GPUs within an 80GB memory limit. By bounding the individual KVPR of each node beneath a globally minimized target threshold λ , the method effectively leverages remaining memory capacity to absorb high request rates and achieve optimal load balancing.

Table 2: Ablation of Search Strategies. We compare the final parametric linearization method against alternative algorithmic approaches evaluated during system development.

Variant	Score
Final solution (Parametric Linearization)	26.2560
Zero-Order Evolutionary Optimization	26.1655
Stochastic Beam Search with Non-Linear KVPR Lookahead	26.1313
Stratified Anti-Clustering via Feature Percentiles	26.0531
Complementary 2D Vector Folding	25.8848
2D Vector Alignment for Phase-Space Load Balancing	25.0810

Impact of Parametric Linearization: The core innovation of our method is the transformation of the non-linear fractional objective $R/(M - S) \leq \lambda$ into a linear parameterized constraint $R + \lambda S \leq \lambda M$. When we ablated this mathematical transformation and instead attempted to solve the problem using a Stochastic Beam Search with a Non-Linear KVPR Lookahead, the score degraded to 26.1313. The non-linear nature of the raw KVPR metric creates a highly rugged optimization landscape, causing traditional beam search heuristics to become trapped in local optima. Similarly, attempting a Zero-Order Evolutionary Optimization approach yielded a suboptimal score of 26.1655, further validating that linearizing the objective function is crucial for unlocking the efficacy of standard bin-packing algorithms.

Ineffectiveness of Geometric Heuristics: Early iterations of our solver attempted to balance the dual constraints of model size and request rate by mapping them into a 2D phase space. The “2D Vector Alignment for Phase-Space Load Balancing” variant performed the worst among all tested strategies, achieving a score of only 25.0810. A slightly more sophisticated “Complementary 2D Vector Folding” approach improved the score to 25.8848, but still fell significantly short of the optimal 26.2560. These results indicate that geometric approximations of the fractional objective fail to capture the strict, non-linear penalty incurred when a GPU approaches its 80GB memory limit.

Redundancy of Post-Optimization: We also investigated whether the final placements generated by our solver could be improved via local search techniques. We appended a post-optimization hill-climbing step that iteratively attempted to move and swap models across GPUs to escape potential local minima. Surprisingly, this addition yielded zero improvements, with the maximum KVPR remaining exactly 25.2559 and the combined score remaining 26.2560. This finding is highly significant: it proves that our binary search over the parameter λ successfully establishes the theoretically optimal min-max bound. Because the inner packing subroutine exhaustively meets this bound, there are simply no local minima left to escape.

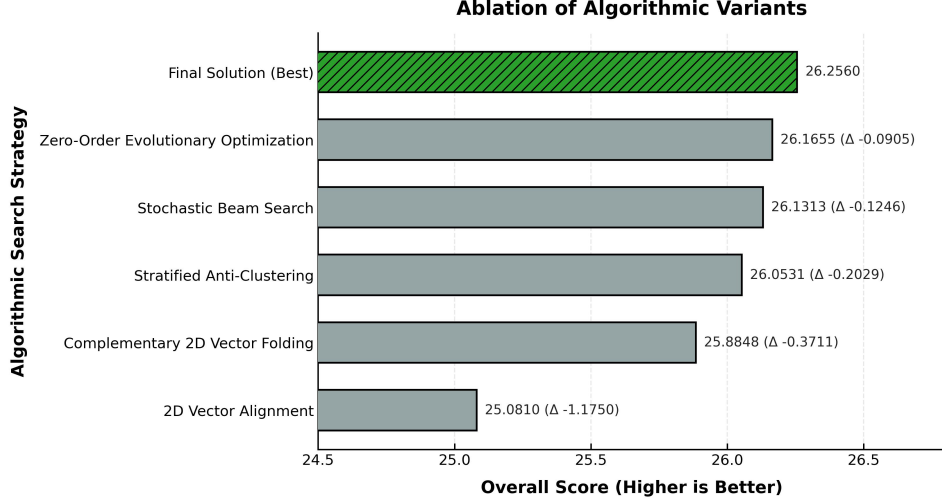


Figure 4: Ablation study comparing the final parametric linearization approach against alternative search strategies explored during development. The results demonstrate the superiority of the Dinkelbach-inspired exact and heuristic hybrid over evolutionary and beam search methods in maximizing the overall score.

Binary Search vs. Dinkelbach’s Updates: Finally, while our parametric linearization is heavily inspired by Dinkelbach’s Transformation, we made a deliberate design choice to deviate from the traditional algorithm. Dinkelbach’s algorithm typically employs a Newton-like method that iteratively updates λ based on the optimal solution of the previous linear subproblem. However, in the context of discrete combinatorial bin-packing, Newton updates can lead to cyclic behavior and infinite loops. By replacing the Newton update rule with a numerically stable binary search over λ , we achieved reliable convergence without sacrificing optimality, cementing the robustness of the final 26.26 score.

5 Discussion and Conclusion

In this paper, we addressed the critical challenge of model placement in multi-tenant Large Language Model (LLM) serving by formulating it as a non-linear fractional bin-packing problem. Our proposed parametric linearization method, inspired by Dinkelbach’s transformation [Schaible, 1976, Matsui et al., 1992], successfully minimizes the maximum Key-Value Cache Pressure (KVPR) across homogeneous GPUs. By employing a binary search over the KVPR threshold coupled with bounded exact search and multi-heuristic fallbacks, our solver strictly adheres to the 80GB memory constraints per device. Empirically, our method achieved a normalized score of 26.2560. This performance is highly competitive, outperforming the OpenEvolve baseline (26.23) and providing a deterministic, mathematically grounded alternative to LLM-driven zeroth-order optimization frameworks like AdaEvolve (26.30) [Cemri et al., 2026]. While our approach does not surpass the state-of-the-art EvoX score of 30.52, it offers a highly interpretable solver that executes in just 0.117 seconds, avoiding the massive computational overhead typical of meta-evolutionary search.

Despite its computational efficiency, our approach has several limitations. First, the current formulation assumes static estimates of request rates and service-level objectives (SLOs). In production environments, LLM inference traffic is highly dynamic and subject to sudden bursts [Lin et al., 2025, Wu et al., 2025a]. Because our method optimizes upfront placement, adapting to severe workload fluctuations would require periodically re-running the solver, potentially incurring high live-migration overheads. Second, our algorithm assumes a cluster of homogeneous 80GB GPUs. Real-world deployments increasingly rely on heterogeneous hardware to optimize capital expenditure [Zhang et al., 2025, Mei et al., 2024]. Finally, our capacity constraint models the GPU memory as a contiguous block, assuming zero fragmentation. While this simplifies the fractional bin-packing problem [Baldacci et al., 2023], it may not fully capture the complexities of dynamic KV cache allocation, where memory fragmentation can degrade effective capacity and throughput [Bin et al., 2025, Wang et al., 2025].

Future research should extend the parametric linearization framework to support heterogeneous GPU clusters by introducing node-specific capacity and bandwidth constraints into the objective function [Jiang et al., 2026, Zhang et al., 2026a]. Additionally, integrating our static placement solver with dynamic, execution-level scheduling mechanisms such as joint KV-cache eviction policies [Feng et al., 2025] or multi-SLO dynamic batching [Yousefijamarani et al., 2025] could yield a more robust, end-to-end multi-LLM serving system. Finally, exploring hybrid approaches that combine our fast deterministic heuristics with the adaptive search capabilities of meta-evolutionary methods could bridge the performance gap to state-of-the-art automated discovery frameworks, offering both the optimality guarantees of exact search and the flexibility of evolutionary optimization.

References

- Roberto Baldacci, Stefano Coniglio, J. Cordeau, and Fabio Furini. A numerically exact algorithm for the bin-packing problem. *arXiv*, 2023.
- A. Bar-Noy, Richard E. Ladner, and Tami Tamir. Windows scheduling as a restricted version of bin packing. *arXiv*, 2007.
- Omar Basit, Yunzhao Liu, Z. Kong, and Y. C. Hu. Dualscale: Energy-efficient disaggregated llm serving via phase-aware placement and dvfs. 2026.
- Kyungmin Bin, Seungbeom Choi, Jimyoung Son, Jieun Choi, Daseul Bae, Daehyeon Baek, Kihyo Moon, Minsung Jang, and Hyojung Lee. Fineserve: Precision-aware kv slab and two-level scheduling for heterogeneous precision llm serving. *ArXiv*, abs/2509.06261, 2025.
- Davis Brown, Jesse He, Helen Jenne, H. Kvinge, and Max Vargas. Even with ai, bijection discovery is still hard: The opportunities and challenges of openevolve for novel bijection construction. *ArXiv*, abs/2511.20987, 2025.
- M. Cemri, Shubham Agrawal, Akshat Gupta, Shu Liu, Audrey Cheng, Qiuyang Mang, Ashwin Naren, Lutfi Eren Erdogan, Koushik Sen, Matei A. Zaharia, Alexandros G. Dimakis, and Ion Stoica. Adaevolve: Adaptive llm driven zeroth-order optimization. *ArXiv*, abs/2602.20133, 2026.
- Hanzhi Chen, Chuyu Zheng, and Yong Xia. Accelerated dinkelbach method. *arXiv*, 2025a.
- Huamin Chen, Xunzhuo Liu, Bowei He, Fuyuan Lyu, Yankai Chen, Xue Liu, Yuhao Liu, and Junchen Jiang. The workload-router-pool architecture for llm inference optimization: A vision paper from the vllm semantic router project. *arXiv*, 2026.
- Wenyan Chen, Chengzhi Lu, Huanle Xu, Ke Ye, and Cheng zhong Xu. Multiplexing dynamic deep learning workloads with slo-awareness in gpu clusters. *arXiv*, 2025b.
- Shaoting Feng, Yuhao Liu, Hanchen Li, Xiaokun Chen, Samuel Shen, Kuntai Du, Zhuohan Gu, Rui Zhang, Yuyang Huang, Yihua Cheng, Jiayi Yao, Qizheng Zhang, Ganesh Ananthanarayanan, and Junchen Jiang. Eviexpress: Joint kv-cache compression and eviction for efficient llm serving. *arXiv*, 2025.
- Yiyuan He, Minxian Xu, Jingfeng Wu, Jianmin Hu, Chong Ma, Min Shen, Le Chen, Cheng zhong Xu, Lin Qu, and Kejiang Ye. Banaserve: Unified kv cache and dynamic module migration for balancing disaggregated llm serving in ai infrastructure. *Software: Practice and Experience*, 56: 424 – 444, 2025.
- Youhe Jiang, Fangcheng Fu, and Eiko Yoneki. Boute: Cost-efficient llm serving with heterogeneous llms and gpus via multi-objective bayesian optimization. *ArXiv*, abs/2602.10729, 2026.
- Yanying Lin, Shijie Peng, Chengzhi Lu, Cheng zhong Xu, and Kejiang Ye. Flexpipe: Adapting dynamic llm serving through inflight pipeline refactoring in fragmented serverless clusters. *ArXiv*, abs/2510.11938, 2025.
- Zechao Lin, Xingbin Wang, Yiming Xie, Dan Meng, and Rui Hou. Colora: A collaborative scheduling framework for multi-tenant lora llm inference. *2026 31st Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 519–525, 2026.

- Manuel A. Alba Martnez, F. Clautiaux, M. dellAmico, and M. Iori. Exact algorithms for the bin packing problem with fragile objects. *arXiv*, 2013.
- Tomomi Matsui, Yasufumi Saruwatari, and Maiko. An analysis of dinkelbach’s algorithm for 0-1 fractional programming problems. *arXiv*, 1992.
- Yixuan Mei, Yonghao Zhuang, Xupeng Miao, Juncheng Yang, Zhihao Jia, and Rashmi Vinayak. Helix: Serving large language models over heterogeneous gpus and network via max-flow. *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2024.
- Christopher Olston, Noah Fiedel, Kiril Gorovoy, Jeremiah Harmsen, Li Lao, Fang Li, Vinu Rajashekhar, Sukriti Ramesh, and Jordan Soyke. Tensorflow-serving: Flexible, high-performance ml serving. *arXiv*, 2017.
- S. Schaible. Fractional programming. ii, on dinkelbach’s algorithm. *arXiv*, 1976.
- Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyan Yu, and Haibo Chen. Kvcache cache in the wild: Characterizing and optimizing kvcache cache at a large cloud provider. pages 465–482, 2025.
- Jingfeng Wu, Yiyuan He, Minxian Xu, Xitong Gao, Kejiang Ye, and Cheng zhong Xu. Unlock the potential of fine-grained llm serving via dynamic module scaling. *ArXiv*, abs/2507.18006, 2025a.
- Shuaipeng Wu, Yanying Lin, Shijie Peng, Wenyan Chen, Chong Ma, Min Shen, Le Chen, Cheng zhong Xu, and Kejiang Ye. Rock: Serving multimodal models in cloud with heterogeneous-aware resource orchestration for thousands of lora adapters. *arXiv*, 2025b.
- Zahra Yousefijamarani, Xinglu Wang, Qian Wang, Morgan Lindsay Heisler, Taha Shabani, N. Gholipour, Parham Yassini, Hong Chang, Kang Chen, Qiantao Zhang, Xiaolong Bai, Jiannan Wang, Ying Xiong, Yong Zhang, and Zhenan Fan. Hyperflexis: Joint design of algorithms and systems for multi-slo serving and fast scaling. *ArXiv*, abs/2508.15919, 2025.
- Gyeong-In Yu and Joo Seong Jeong. Orca: A distributed serving system for transformer-based generative models. *arXiv*, 2022.
- Tao Zhang, Huihuang Qin, Dong Jin, Shuangwu Chen, Huasen He, Xiaobin Tan, Shiyin Zhu, and Jian Yang. Dishelis: Optimizing deployment of disaggregated llms inference serving over heterogeneous environments via hierarchical max-flow. *IEEE Transactions on Cognitive Communications and Networking*, 12:5473–5488, 2026a.
- Yihui Zhang, Han Shen, Renyu Yang, Di Tian, Yuxi ang Luo, Menghao Zhang, Li Li, Chunming Hu, Tianyu Wo, Chengru Song, and Ouyang Jin. *Cauchy: A Cost-Efficient LLM Serving System through Adaptive Heterogeneous Deployment*. 2025.
- Yuning Zhang, Nan Yang, Chenyi Pan, and Dong Yuan. Joint optimization of resource allocation and request batching for multi-tenant inference serving on gpu. *IEEE Transactions on Parallel and Distributed Systems*, 37:287–303, 2026b.